11.4 Splay Trees

The next search tree structure we study is known as a a ***splay tree***. This structure is conceptually quite different from the other balanced search trees we discuss in this chapter, for a splay tree does not strictly enforce a logarithmic upper bound on the height of the tree. In fact, there are no additional height, balance, or other auxiliary data associated with the nodes of this tree.

The efficiency of splay trees is due to a certain move-to-root operation, called ***splaying***, that is performed at the bottommost position p reached during every insertion, deletion, or even a search. (In essence, this is a tree variant of the move-to-front heuristic that we explored for lists in Section 7.6.2.) Intuitively, a splay operation causes more frequently accessed elements to remain nearer to the root, thereby reducing the typical search times. The surprising thing about splaying is that it allows us to guarantee a logarithmic amortized running time, for insertions, deletions, and searches.

11.4.1 Splaying

Given a node x of a binary search tree T, we ***splay*** x by moving x to the root of T through a sequence of restructurings. The particular restructurings we perform are important, for it is not sufficient to move x to the root of T by just any sequence of restructurings. The specific operation we perform to move x up depends upon the relative positions of x, its parent y, and (if it exists) x's grandparent z. There are three cases that we consider.

***zig-zig*:** The node x and its parent y are both left children or both right children. (See Figure 11.16.) We promote x, making y a child of x and z a child of y, while maintaining the inorder relationships of the nodes in T.

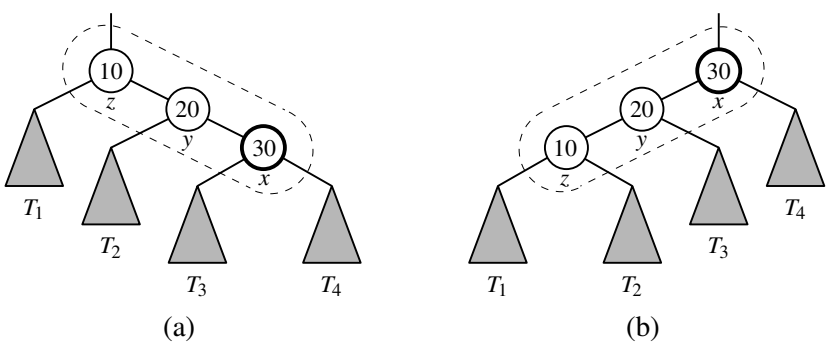


Figure 11.16: Zig-zig: (a) before; (b) after. There is another symmetric configuration where x and y are left children.

***zig-zag*:** One of x and y is a left child and the other is a right child. (See Figure 11.17.) In this case, we promote x by making x have y and z as its children, while maintaining the inorder relationships of the nodes in T.

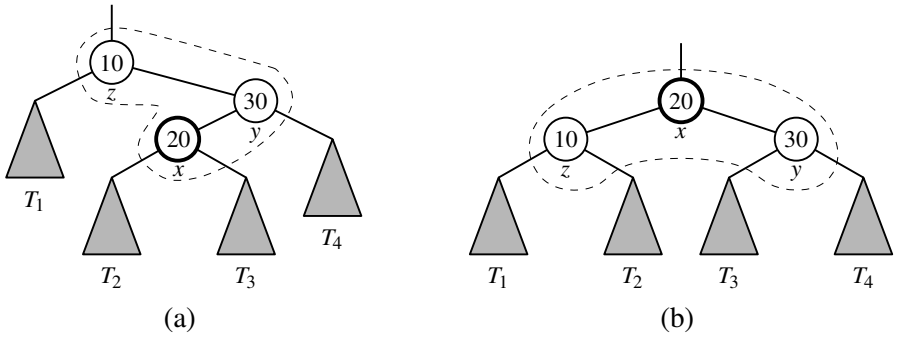


Figure 11.17: Zig-zag: (a) before; (b) after. There is another symmetric configuration where x is a right child and y is a left child.

***zig*:** x does not have a grandparent. (See Figure 11.18.) In this case, we perform a single rotation to promote x over y, making y a child of x, while maintaining the relative inorder relationships of the nodes in T.

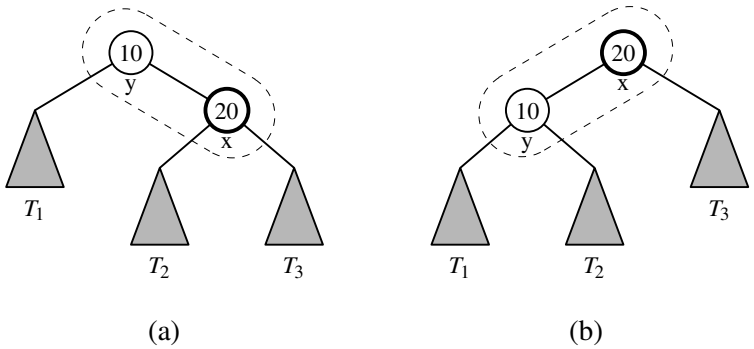
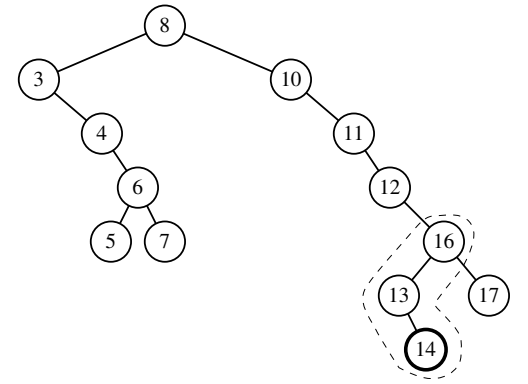
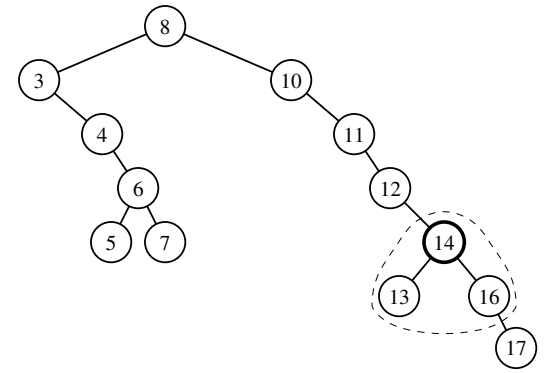


Figure 11.18: Zig: (a) before; (b) after. There is another symmetric configuration where x is originally a left child of y.

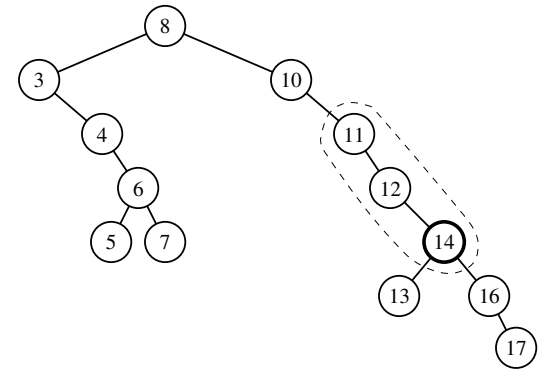
We perform a zig-zig or a zig-zag when x has a grandparent, and we perform a zig when x has a parent but not a grandparent. A ***splaying*** step consists of repeating these restructurings at x until x becomes the root of T. An example of the splaying of a node is shown in Figures 11.19 and 11.20.



(a)

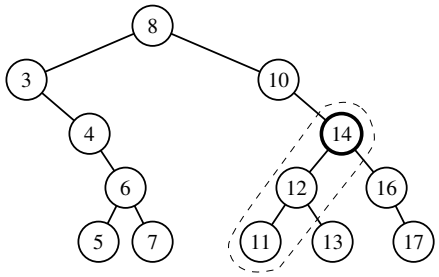


(b)

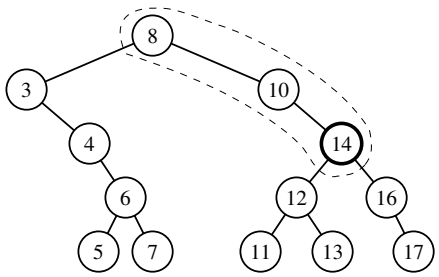


(c)

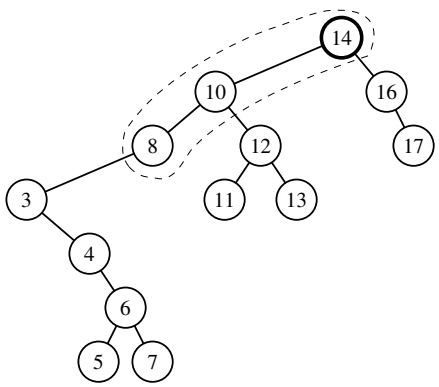
Figure 11.19: Example of splaying a node: (a) splaying the node storing 14 starts with a zig-zag; (b) after the zig-zag; (c) the next step will be a zig-zig. (Continues in Figure 11.20.)



(d)



(e)



(f)

Figure 11.20: Example of splaying a node:(d) after the zig-zig; (e) the next step is again a zig-zig; (f) after the zig-zig. (Continued from Figure 11.19.)

11.4.2 When to Splay

The rules that dictate when splaying is performed are as follows:

- When searching for key k, if k is found at position p, we splay p, else we splay the leaf position at which the search terminates unsuccessfully. For example, the splaying in Figures 11.19 and 11.20 would be performed after searching successfully for key 14 or unsuccessfully for key 15.
- When inserting key k, we splay the newly created internal node where k gets inserted. For example, the splaying in Figures 11.19 and 11.20 would be performed if 14 were the newly inserted key. We show a sequence of insertions in a splay tree in Figure 11.21.

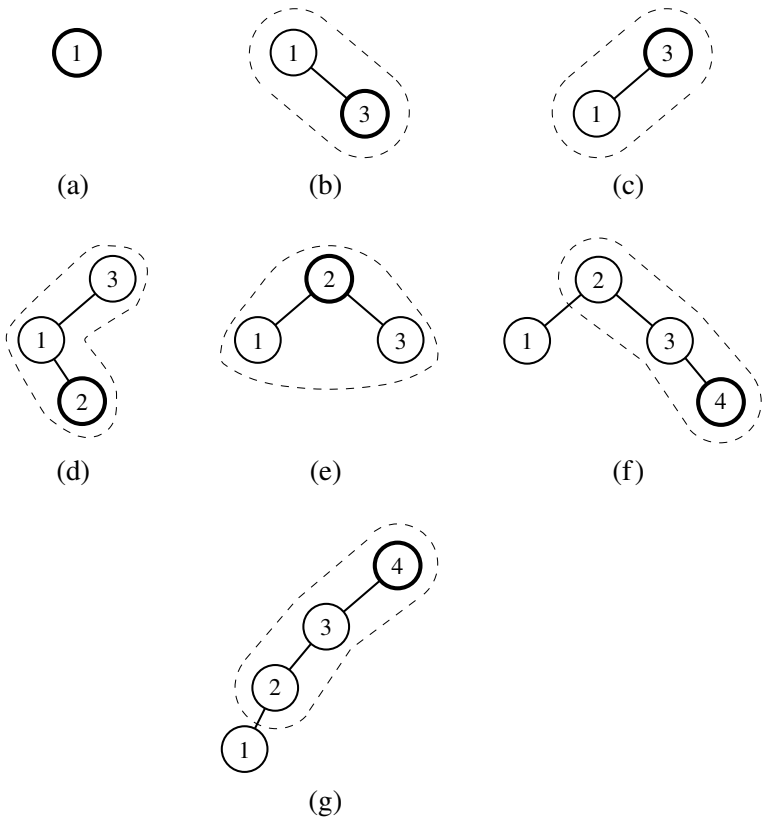


Figure 11.21: A sequence of insertions in a splay tree: (a) initial tree; (b) after inserting 3, but before a zig step; (c) after splaying; (d) after inserting 2, but before a zig-zag step; (e) after splaying; (f) after inserting 4, but before a zig-zig step; (g) after splaying.

- When deleting a key k, we splay the position p that is the parent of the removed node; recall that by the removal algorithm for binary search trees, the removed node may be that originally containing k, or a descendant node with a replacement key. An example of splaying following a deletion is shown in Figure 11.22.

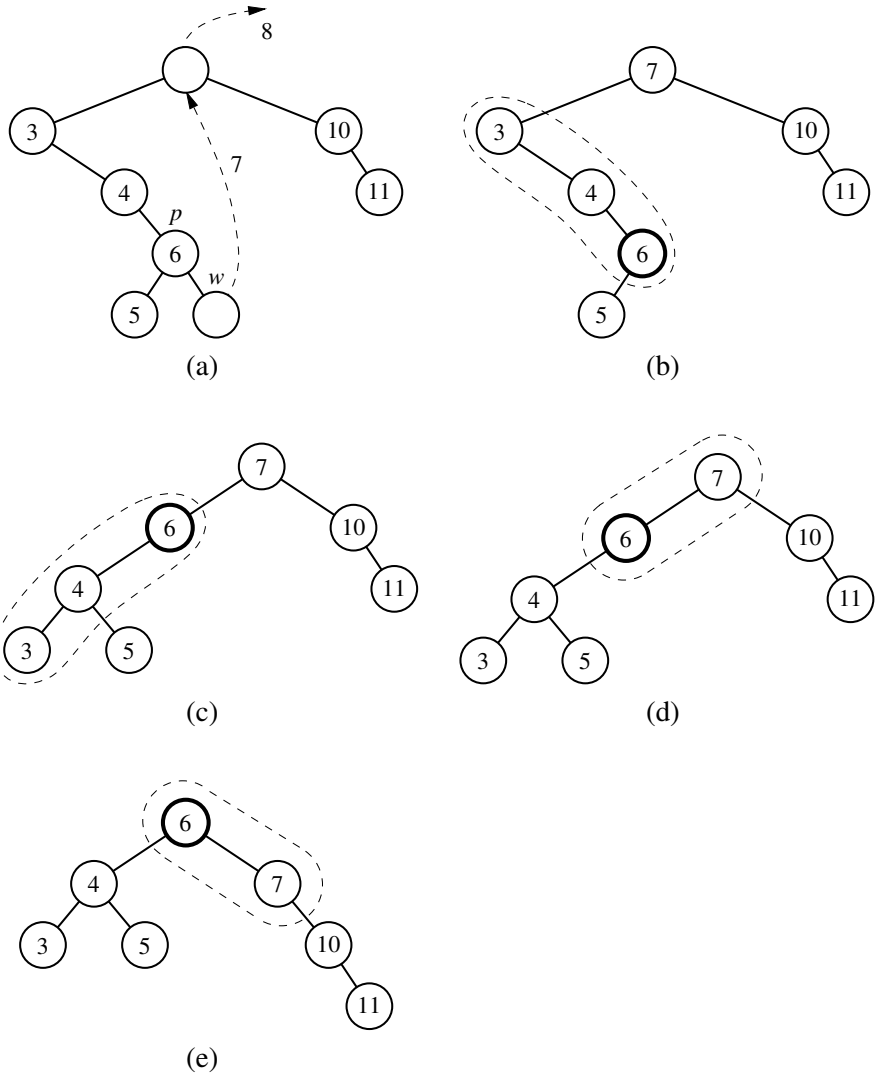


Figure 11.22: Deletion from a splay tree: (a) the deletion of 8 from the root node is performed by moving to the root the key of its inorder predecessor w, deleting w, and splaying the parent p of w; (b) splaying p starts with a zig-zig; (c) after the zig-zig; (d) the next step is a zig; (e) after the zig.

11.4.3 Python Implementation

Although the mathematical analysis of a splay tree's performance is complex (see Section 11.4.4), the *implementation* of splay trees is a rather simple adaptation to a standard binary search tree. Code Fragment 11.14 provides a complete implementation of a SplayTreeMap class, based upon the underlying TreeMap class and use of the balancing framework described in Section 11.2.1. It is important to note that our original TreeMap class makes calls to the _rebalance_access method, not just from within the __getitem__ method, but also during __setitem__ when modifying the value associated with an existing key, and after any map operations that result in a failed search.

```
1  class SplayTreeMap(TreeMap):
2    """Sorted map implementation using a splay tree."""
3    #------------------------------ splay operation ------------------------------
4    def _splay(self, p):
5      while p != self.root( ):
6        parent = self.parent(p)
7        grand = self.parent(parent)
8        if grand is None:
9          # zig case
10         self._rotate(p)
11       elif (parent == self.left(grand)) == (p == self.left(parent)):
12         # zig-zig case
13         self._rotate(parent)                    # move PARENT up
14         self._rotate(p)                         # then move p up
15       else:
16         # zig-zag case
17         self._rotate(p)                         # move p up
18         self._rotate(p)                         # move p up again
19
20   #---------------------------- override balancing hooks ----------------------------
21   def _rebalance_insert(self, p):
22     self._splay(p)
23
24   def _rebalance_delete(self, p):
25     if p is not None:
26       self._splay(p)
27
28   def _rebalance_access(self, p):
29     self._splay(p)
```

Code Fragment 11.14: A complete implementation of the SplayTreeMap class.

11.4.4 Amortized Analysis of Splaying ⋆

After a zig-zig or zig-zag, the depth of position p decreases by two, and after a zig the depth of p decreases by one. Thus, if p has depth d, splaying p consists of a sequence of $\lfloor d/2 \rfloor$ zig-zigs and/or zig-zags, plus one final zig if d is odd. Since a single zig-zig, zig-zag, or zig affects a constant number of nodes, it can be done in $O(1)$ time. Thus, splaying a position p in a binary search tree T takes time $O(d)$, where d is the depth of p in T. In other words, the time for performing a splaying step for a position p is asymptotically the same as the time needed just to reach that position in a top-down search from the root of T.

Worst-Case Time

In the worst case, the overall running time of a search, insertion, or deletion in a splay tree of height h is $O(h)$, since the position we splay might be the deepest position in the tree. Moreover, it is possible for h to be as large as n, as shown in Figure 11.21. Thus, from a worst-case point of view, a splay tree is not an attractive data structure.

In spite of its poor worst-case performance, a splay tree performs well in an amortized sense. That is, in a sequence of intermixed searches, insertions, and deletions, each operation takes on average logarithmic time. We perform the amortized analysis of splay trees using the accounting method.

Amortized Performance of Splay Trees

For our analysis, we note that the time for performing a search, insertion, or deletion is proportional to the time for the associated splaying. So let us consider only splaying time.

Let T be a splay tree with n keys, and let w be a node of T. We define the ***size*** $n(w)$ of w as the number of nodes in the subtree rooted at w. Note that this definition implies that the size of a nonleaf node is one more than the sum of the sizes of its children. We define the ***rank*** $r(w)$ of a node w as the logarithm in base 2 of the size of w, that is, $r(w) = \log(n(w))$. Clearly, the root of T has the maximum size, n, and the maximum rank, $\log n$, while each leaf has size 1 and rank 0.

We use cyber-dollars to pay for the work we perform in splaying a position p in T, and we assume that one cyber-dollar pays for a zig, while two cyber-dollars pay for a zig-zig or a zig-zag. Hence, the cost of splaying a position at depth d is d cyber-dollars. We keep a virtual account storing cyber-dollars at each position of T. Note that this account exists only for the purpose of our amortized analysis, and does not need to be included in a data structure implementing the splay tree T.

An Accounting Analysis of Splaying

When we perform a splaying, we pay a certain number of cyber-dollars (the exact value of the payment will be determined at the end of our analysis). We distinguish three cases:

- If the payment is equal to the splaying work, then we use it all to pay for the splaying.
- If the payment is greater than the splaying work, we deposit the excess in the accounts of several nodes.
- If the payment is less than the splaying work, we make withdrawals from the accounts of several nodes to cover the deficiency.

We show below that a payment of $O(\log n)$ cyber-dollars per operation is sufficient to keep the system working, that is, to ensure that each node keeps a nonnegative account balance.

An Accounting Invariant for Splaying

We use a scheme in which transfers are made between the accounts of the nodes to ensure that there will always be enough cyber-dollars to withdraw for paying for splaying work when needed.

In order to use the accounting method to perform our analysis of splaying, we maintain the following invariant:

> ***Before and after a splaying, each node*** w ***of*** T ***has*** $r(w)$ ***cyber-dollars in its account.***

Note that the invariant is "financially sound," since it does not require us to make a preliminary deposit to endow a tree with zero keys.

Let $r(T)$ be the sum of the ranks of all the nodes of T. To preserve the invariant after a splaying, we must make a payment equal to the splaying work plus the total change in $r(T)$. We refer to a single zig, zig-zig, or zig-zag operation in a splaying as a splaying ***substep***. Also, we denote the rank of a node w of T before and after a splaying substep with $r(w)$ and $r'(w)$, respectively. The following proposition gives an upper bound on the change of $r(T)$ caused by a single splaying substep. We will repeatedly use this lemma in our analysis of a full splaying of a node to the root.

Proposition 11.3: *Let δ be the variation of $r(T)$ caused by a single splaying substep (a zig, zig-zig, or zig-zag) for a node x in T. We have the following:*

- $\delta \leq 3(r'(x) - r(x)) - 2$ *if the substep is a zig-zig or zig-zag.*
- $\delta \leq 3(r'(x) - r(x))$ *if the substep is a zig.*

Justification: We use the fact (see Proposition B.1, Appendix A) that, if $a > 0$, $b > 0$, and $c > a + b$,

$$\log a + \log b < 2\log c - 2. \tag{11.6}$$

Let us consider the change in $r(T)$ caused by each type of splaying substep.

***zig-zig*:** (Recall Figure 11.16.) Since the size of each node is one more than the size of its two children, note that only the ranks of x, y, and z change in a zig-zig operation, where y is the parent of x and z is the parent of y. Also, $r'(x) = r(z)$, $r'(y) \leq r'(x)$, and $r(x) \leq r(y)$. Thus,

$$\begin{aligned} \delta &= r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \\ &= r'(y) + r'(z) - r(x) - r(y) \\ &\leq r'(x) + r'(z) - 2r(x). \end{aligned} \tag{11.7}$$

Note that $n(x) + n'(z) < n'(x)$. Thus, $r(x) + r'(z) < 2r'(x) - 2$, as per Formula 11.6; that is,

$$r'(z) < 2r'(x) - r(x) - 2.$$

This inequality and Formula 11.7 imply

$$\begin{aligned} \delta &\leq r'(x) + (2r'(x) - r(x) - 2) - 2r(x) \\ &\leq 3(r'(x) - r(x)) - 2. \end{aligned}$$

***zig-zag*:** (Recall Figure 11.17.) Again, by the definition of size and rank, only the ranks of x, y, and z change, where y denotes the parent of x and z denotes the parent of y. Also, $r(x) < r(y) < r(z) = r'(x)$. Thus,

$$\begin{aligned} \delta &= r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \\ &= r'(y) + r'(z) - r(x) - r(y) \\ &\leq r'(y) + r'(z) - 2r(x). \end{aligned} \tag{11.8}$$

Note that $n'(y) + n'(z) < n'(x)$; hence, $r'(y) + r'(z) < 2r'(x) - 2$, as per Formula 11.6. Thus,

$$\begin{aligned} \delta &\leq 2r'(x) - 2 - 2r(x) \\ &= 2(r'(x) - r(x)) - 2 \leq 3(r'(x) - r(x)) - 2. \end{aligned}$$

***zig*:** (Recall Figure 11.18.) In this case, only the ranks of x and y change, where y denotes the parent of x. Also, $r'(y) \leq r(y)$ and $r'(x) \geq r(x)$. Thus,

$$\begin{aligned} \delta &= r'(y) + r'(x) - r(y) - r(x) \\ &\leq r'(x) - r(x) \\ &\leq 3(r'(x) - r(x)). \end{aligned}$$

■

Proposition 11.4: *Let T be a splay tree with root t, and let Δ be the total variation of $r(T)$ caused by splaying a node x at depth d. We have*

$$\Delta \leq 3(r(t) - r(x)) - d + 2.$$

Justification: Splaying node x consists of $c = \lceil d/2 \rceil$ splaying substeps, each of which is a zig-zig or a zig-zag, except possibly the last one, which is a zig if d is odd. Let $r_0(x) = r(x)$ be the initial rank of x, and for $i = 1, \ldots, c$, let $r_i(x)$ be the rank of x after the ith substep and δ_i be the variation of $r(T)$ caused by the ith substep. By Proposition 11.3, the total variation Δ of $r(T)$ caused by splaying x is

$$\begin{aligned}
\Delta &= \sum_{i=1}^{c} \delta_i \\
&\leq 2 + \sum_{i=1}^{c} 3(r_i(x) - r_{i-1}(x)) - 2 \\
&= 3(r_c(x) - r_0(x)) - 2c + 2 \\
&\leq 3(r(t) - r(x)) - d + 2.
\end{aligned}$$

■

By Proposition 11.4, if we make a payment of $3(r(t) - r(x)) + 2$ cyber-dollars towards the splaying of node x, we have enough cyber-dollars to maintain the invariant, keeping $r(w)$ cyber-dollars at each node w in T, and pay for the entire splaying work, which costs d cyber-dollars. Since the size of the root t is n, its rank $r(t) = \log n$. Given that $r(x) \geq 0$, the payment to be made for splaying is $O(\log n)$ cyber-dollars. To complete our analysis, we have to compute the cost for maintaining the invariant when a node is inserted or deleted.

When inserting a new node w into a splay tree with n keys, the ranks of all the ancestors of w are increased. Namely, let $w_0, w_i, \ldots, w_d$ be the ancestors of w, where $w_0 = w$, w_i is the parent of w_{i-1}, and w_d is the root. For $i = 1, \ldots, d$, let $n'(w_i)$ and $n(w_i)$ be the size of w_i before and after the insertion, respectively, and let $r'(w_i)$ and $r(w_i)$ be the rank of w_i before and after the insertion. We have

$$n'(w_i) = n(w_i) + 1.$$

Also, since $n(w_i) + 1 \leq n(w_{i+1})$, for $i = 0, 1, \ldots, d-1$, we have the following for each i in this range:

$$r'(w_i) = \log(n'(w_i)) = \log(n(w_i) + 1) \leq \log(n(w_{i+1})) = r(w_{i+1}).$$

Thus, the total variation of $r(T)$ caused by the insertion is

$$\begin{aligned}
\sum_{i=1}^{d} \big(r'(w_i) - r(w_i)\big) &\leq r'(w_d) + \sum_{i=1}^{d-1} (r(w_{i+1}) - r(w_i)) \\
&= r'(w_d) - r(w_0) \\
&\leq \log n.
\end{aligned}$$

Therefore, a payment of $O(\log n)$ cyber-dollars is sufficient to maintain the invariant when a new node is inserted.

When deleting a node w from a splay tree with n keys, the ranks of all the ancestors of w are decreased. Thus, the total variation of $r(T)$ caused by the deletion is negative, and we do not need to make any payment to maintain the invariant when a node is deleted. Therefore, we may summarize our amortized analysis in the following proposition (which is sometimes called the "balance proposition" for splay trees):

Proposition 11.5: *Consider a sequence of m operations on a splay tree, each one a search, insertion, or deletion, starting from a splay tree with zero keys. Also, let n_i be the number of keys in the tree after operation i, and n be the total number of insertions. The total running time for performing the sequence of operations is*

$$O\left(m + \sum_{i=1}^{m} \log n_i\right),$$

which is $O(m \log n)$.

In other words, the amortized running time of performing a search, insertion, or deletion in a splay tree is $O(\log n)$, where n is the size of the splay tree at the time. Thus, a splay tree can achieve logarithmic-time amortized performance for implementing a sorted map ADT. This amortized performance matches the worst-case performance of AVL trees, $(2,4)$ trees, and red-black trees, but it does so using a simple binary tree that does not need any extra balance information stored at each of its nodes. In addition, splay trees have a number of other interesting properties that are not shared by these other balanced search trees. We explore one such additional property in the following proposition (which is sometimes called the "Static Optimality" proposition for splay trees):

Proposition 11.6: *Consider a sequence of m operations on a splay tree, each one a search, insertion, or deletion, starting from a splay tree T with zero keys. Also, let $f(i)$ denote the number of times the entry i is accessed in the splay tree, that is, its frequency, and let n denote the total number of entries. Assuming that each entry is accessed at least once, then the total running time for performing the sequence of operations is*

$$O\left(m + \sum_{i=1}^{n} f(i) \log(m/f(i))\right).$$

We omit the proof of this proposition, but it is not as hard to justify as one might imagine. The remarkable thing is that this proposition states that the amortized running time of accessing an entry i is $O(\log(m/f(i)))$.